STATISTICS WITH R PROGRAMMING

ASSIGNMENT 3 – CO3

Design and Develop an Integrated R Data-Frame Management Pipeline for a Multi-Branch Clinic Records Analytics System

Submission detail	Information
Course	Statistics with R Programming
Assignment outcome	CO3 – Design a normalised data frame and write functions in R to implement simple operations with dataset
Nature of assignment	Pair work – 2 students per group
Student A name / ID	P.VIGNESH
Student B name / ID	-
Class / Department	BTech AIML
Faculty	Dr.S.Radhika
Submission date	9-11-2026

Abstract

This assignment develops a reproducible R data-frame pipeline for a fictional multi-branch clinic network. The pipeline brings together patient, doctor, appointment, laboratory-test and billing records that may arrive as separate CSV or delimited files. It demonstrates data-frame creation, inspection, selection, modification, cleaning, reshaping, merging, file import/export, reusable functions, testing and performance evaluation. The solution uses synthetic identifiers and values only. It also explains common risks such as duplicate keys, missing values, type conversion, unmatched joins and accidental file overwriting.

Problem statement

The clinic initially handles approximately 5,000 appointments per month across 10 branches, with a future requirement of 500,000 appointments per month across 100 branches. The system therefore needs clear keys, validated data types, traceable cleaning rules, safe joins and repeatable outputs.

Scope and assumptions

- The example uses synthetic data generated within R; names and identifiers are fictional.
- PatientID, DoctorID, AppointmentID, LabID and BillID are treated as stable identifiers.
- Appointments are the central fact table. Patients and doctors are descriptive tables; lab tests and bills are child records.
- A single appointment may have zero, one or multiple laboratory tests and may have one or more billing line items. Therefore, joins to child tables can legitimately multiply appointment rows.
- The code uses base R and reshape2. Install reshape2 once with install.packages('reshape2') if it is not already installed.
- Benchmark execution times must be measured on the student's own computer. No machine-specific timing values are invented in this report.

Contents

1. Task 1 – Data enquiry and architecture
2. Task 2 – Source data frames
3. Task 3 – Manipulation and summarisation
4. Task 4 – Editing and cleaning
5. Task 5 – Melting and casting
6. Task 6 – Multi-stage merging
7. Task 7 – File reading and writing
8. Task 8 – Reusable function library

9. Task 9 – Integrated testing
10. Task 10 – Efficiency and maintainability
11. Task 11 – Changed requirement
12. Task 12 – Documentation and presentation
13. Consolidated R script
14. Conclusion
15. References
16. Contribution record

Overall system architecture



Figure 1. End-to-end pipeline. Each stage writes a controlled output and records counts or validation results so that errors can be traced.

Normalised data model and relationships

Data frame	Primary key	Important foreign key(s)	Purpose
Patients	PatientID	—	One row per patient
Doctors	DoctorID	DepartmentID	One row per doctor
Appointments	AppointmentID	PatientID, DoctorID, BranchID	One row per appointment
LabTests	LabID	AppointmentID	One row per laboratory test
Billing	BillID	AppointmentID	One row per billing line
Branches	BranchID	—	One row per clinic branch
Departments	DepartmentID	—	One row per department

Normalisation note: patient details are stored once in Patients rather than repeated in every appointment. Doctor and department details are stored separately. Appointment-specific facts remain in Appointments. Laboratory and billing records use their own primary keys and refer to AppointmentID. This reduces update anomalies and makes relationships explicit.

TASK 1 – Formulate the Data Enquiry and Create the Data-Frame Architecture Blueprint (4 Marks)

Objective: Translate the clinic scenario into a schema, relationship plan, testable questions and acceptance criteria.

Proposed enquiry questions: (1) Does explicit colClasses import preserve identifiers as character? (2) Which join preserves every appointment? (3) Do melt/cast transformations preserve totals? (4) Can exported files be read back without changing values? (5) How does processing time change from 5,000 to 500,000 appointments?

Shared group work

- Identify the five required subject areas: patients, doctors, appointments, laboratory tests and billing.
- Define candidate keys and foreign-key relationships before coding.
- Set up a reproducible R session and decide which operations will answer the enquiry questions.
- Use synthetic data only and document all assumptions.

Student A – individual responsibility

- Design Patients, Appointments and Billing schemas with field names, types and keys.
- Map business requirements to columns and explain why each entity is stored separately.

Student B – individual responsibility

- Design Doctors and LabTests schemas and the relationship map.
- Identify risks such as missing foreign keys, duplicate identifiers and one-to-many joins.

R implementation

```
# Reproducible session setup
set.seed(2026)
options(stringsAsFactors = FALSE)
project_dir <- "clinic_pipeline"
dir.create(project_dir, showWarnings = FALSE)
dir.create(file.path(project_dir, "input"), showWarnings = FALSE)
dir.create(file.path(project_dir, "output"), showWarnings = FALSE)
required_packages <- c("reshape2")
missing_packages <- required_packages[
  !vapply(required_packages, requireNamespace, logical(1), quietly = TRUE)
]
if (length(missing_packages)) install.packages(missing_packages)
library(reshape2)
R.version.string
sessionInfo()
```

Expected evidence / interpretation

Deliver a schema blueprint, relationship map, reproducible setup, enquiry questions and acceptance criteria.

Acceptance criterion	Verification method	Pass condition
Correctness	Known test records and manual checks	Expected values match
Consistency	Schema and category validation	No unapproved type/category
Traceability	Input/output count log and rejected rows	Every exclusion is explained
Reproducibility	Run twice with same seed and inputs	Same values and dimensions
Efficiency	Benchmark at 5,000 and 500,000 rows	Time and object size recorded
Safe output	Timestamped names and file.exists()	Existing file is not overwritten

TASK 2 – Create and Validate the Source Data Frames (4 Marks)

Objective: Build realistic synthetic data frames with explicit types and controlled defects, then inspect them before processing.

Character identifiers are recommended even when they contain digits. Treating an ID as numeric can remove leading zeroes or cause unwanted numeric operations. Dates should be converted explicitly after import.

Shared group work

- Create at least five related data frames and keep identifiers as character values.
- Introduce documented defects for testing: missing phone, duplicate patient, inconsistent date, and unmatched doctor or appointment reference.
- Inspect each frame using str(), summary(), head(), dim(), names() and class().
- Validate required columns and key uniqueness before any merge.

Student A – individual responsibility

- Construct Patients and Appointments and inject controlled defects.
- Explain each defect and its expected treatment.

Student B – individual responsibility

- Construct Doctors, LabTests and Billing.
- Implement schema, type, missing-value and identifier checks.

R implementation

```
# Synthetic source data
Patients <- data.frame(
  PatientID = c("P001","P002","P003","P004","P004"),
  PatientName = c("Asha","Bala","Charan","Divya","Divya"),
  Age = c(28L, 42L, 35L, 51L, 51L),
  Gender = factor(c("F","M","M","F","F")),
  Phone = c("9000000001", NA, "9000000003", "9000000004", "9000000004"),
  stringsAsFactors = FALSE
)
```

```
Doctors <- data.frame(
  DoctorID = c("D01","D02","D03"),
  DoctorName = c("Dr. Rao","Dr. Meena","Dr. Kumar"),
  DepartmentID = c("DEP1","DEP2","DEP1"),
  stringsAsFactors = FALSE
)
Appointments <- data.frame(
  AppointmentID = sprintf("A%03d", 1:6),
  PatientID = c("P001","P002","P003","P004","P001","P999"),
  DoctorID = c("D01","D02","D01","D03","D02","D99"),
  BranchID = c("B01","B01","B02","B02","B01","B03"),
  AppointmentDate = c("2026-01-03","03/01/2026","2026-01-05",
                      "2026-01-06","2026-01-07","2026-01-08"),
  Status = c("Completed","completed","Booked","Completed","Cancelled","Completed"),
  Fee = c(500,600,450,700,500,300),
  stringsAsFactors = FALSE
)
LabTests <- data.frame(
  LabID = c("L001","L002","L003"),
  AppointmentID = c("A001","A001","A003"),
  TestName = c("CBC","Glucose","Lipid"),
  ResultValue = c(12.5, 98, 180)
)
Billing <- data.frame(
  BillID = c("BL001","BL002","BL003","BL004"),
  AppointmentID = c("A001","A002","A003","A004"),
  Amount = c(500,600,450,700),
  PaymentStatus = c("Paid","Pending","Paid","Paid")
)
Branches <- data.frame(BranchID=c("B01","B02","B03"),
                       BranchName=c("North","South","East"))
Departments <- data.frame(DepartmentID=c("DEP1","DEP2"),
                          DepartmentName=c("General","Cardiology"))

# Structure inspection
lapply(list(Patients=Patients, Doctors=Doctors,
            Appointments=Appointments, LabTests=LabTests,
            Billing=Billing), function(x) {
  list(class=class(x), dimensions=dim(x), names=names(x),
       structure=str(x), summary=summary(x), preview=head(x))
})
```

Expected evidence / interpretation

Include executable creation code, structure outputs, defect register and validation results.

Defect injected	Location	Expected handling

Duplicate PatientID P004	Patients	Flag and resolve using a documented survivor rule
Missing phone	Patients	Retain patient; mark contact field missing
Mixed date formats	Appointments	Parse supported formats; flag unparseable values
Lowercase status	Appointments	Standardise case and trim whitespace
Unknown PatientID P999	Appointments	Retain as exception; do not silently discard
Unknown DoctorID D99	Appointments	Place in unmatched-reference report

TASK 3 – Develop a Data-Frame Manipulation and Summarisation Module (4 Marks)

Objective: Demonstrate selection, modification, ordering and grouped summaries using base R operations.

Use drop = FALSE when a one-column subset must remain a data frame. Use [[ ]] when a single column object is required. Row names are labels, not reliable business keys; use explicit ID columns.

Shared group work

- Use $, [ ], [[ ]] and subset() for access and filtering.
- Add, update and remove columns and rows; demonstrate rbind() and cbind().
- Use order(), aggregate(), tapply(), table(), apply(), with() and within().
- Test empty selections, absent columns and out-of-range row positions.

Student A – individual responsibility

- Lead row/column selection and modification and show before/after examples.
- Explain whether each selection returns a vector, data frame or list.

Student B – individual responsibility

- Lead ordering and grouped summaries.
- Compare aggregate() and tapply() and verify a small total manually.

R implementation

```
# Access methods
Appointments$Fee
Appointments[1:3, c("AppointmentID","PatientID"), drop = FALSE]
Appointments[["Status"]]
subset(Appointments, Fee >= 500)

# Add, modify and remove columns
Appointments$Tax <- round(Appointments$Fee * 0.05, 2)
Appointments$Status <- trimws(tolower(Appointments$Status))
Appointments$TemporaryFlag <- TRUE
Appointments$TemporaryFlag <- NULL
```

```
# Order by date then fee after date parsing
Appointments$AppointmentDate2 <- as.Date(
  Appointments$AppointmentDate, format = "%Y-%m-%d"
)
Appointments_sorted <- Appointments[order(Appointments$Fee,
                                          decreasing = TRUE), ]

# Summaries
aggregate(Fee ~ BranchID, data = Appointments, FUN = sum)
tapply(Appointments$Fee, Appointments$BranchID, sum, na.rm = TRUE)
table(Appointments$Status)
with(Appointments, mean(Fee, na.rm = TRUE))
Appointments <- within(Appointments, FeeBand <- ifelse(Fee >= 500, "High", "Standard"))

# Combine compatible data frames
extra_row <- data.frame(AppointmentID="A007", PatientID="P002",
  DoctorID="D01", BranchID="B03", AppointmentDate="2026-01-09",
  Status="booked", Fee=400)
Appointments_extended <- rbind(Appointments, extra_row)
Appointments_tagged <- cbind(Appointments, SourceFile="branch01.csv")

# Boundary checks
empty_result <- subset(Appointments, Fee < 0)
stopifnot(nrow(empty_result) == 0)
if (!"UnknownColumn" %in% names(Appointments)) message("Required column absent")
safe_row <- function(x, i) {
  if (i < 1 || i > nrow(x)) return(x[0, , drop=FALSE])
  x[i, , drop=FALSE]
}
safe_row(Appointments, 999)
```

Expected evidence / interpretation

Show code, sample output, summary comparison and boundary tests.

Operation	Example	Typical result
$	df$Fee	Vector
[ ]	df[, c('Fee','Status'), drop=FALSE]	Data frame
[[ ]]	df[['Fee']]	Single column vector
subset()	subset(df, Fee > 500)	Data frame
aggregate()	aggregate(Fee ~ BranchID, ...)	Grouped data frame
tapply()	tapply(Fee, BranchID, sum)	Named vector

TASK 4 – Build a Data-Frame Editing and Cleaning Module (4 Marks)

Objective: Create reusable and traceable cleaning rules for missing values, duplicates, inconsistent names, categories and dates.

Automated cleaning is insufficient because unusual but valid values may be mistaken for errors. For example, two records with the same patient name may belong to different people. A safe pipeline flags ambiguous records for human review instead of deleting them automatically.

Shared group work

- Use is.na(), complete.cases(), na.omit(), duplicated() and unique().
- Use imputation only where justified; do not invent missing patient identity or contact data.
- Standardise names and categories and convert types explicitly.
- Keep records requiring manual review in a separate exception set and compare row counts.

Student A – individual responsibility

- Lead missing-value, duplicate and column-name cleaning.
- Show before-and-after examples and explain the survivor rule.

Student B – individual responsibility

- Lead date/category standardisation and manual-review flags.
- Test valid, invalid and boundary dates and reconcile counts.

R implementation

```
clean_names <- function(x) {
 names(x) <- make.names(trimws(names(x)), unique = TRUE)
 x
}
clean_patients <- function(x) {
 stopifnot(is.data.frame(x), "PatientID" %in% names(x))
 before <- nrow(x)
 x <- clean_names(x)
 x$PatientID <- trimws(as.character(x$PatientID))
 x$PatientName <- trimws(as.character(x$PatientName))
 x$Age <- as.integer(x$Age)
 x$Phone <- as.character(x$Phone)
 x$DuplicateID <- duplicated(x$PatientID) |
                  duplicated(x$PatientID, fromLast = TRUE)
 # Keep first ID occurrence for the clean master; preserve all flagged rows.
 rejected <- x[x$DuplicateID, , drop=FALSE]
 cleaned <- x[!duplicated(x$PatientID), , drop=FALSE]
 list(data=cleaned, rejected=rejected,
     counts=list(input=before, output=nrow(cleaned),
                  flagged=nrow(rejected)))
}
```

```
parse_clinic_date <- function(x) {
  x <- trimws(as.character(x))
  d1 <- as.Date(x, format="%Y-%m-%d")
  d2 <- as.Date(x, format="%d/%m/%Y")
  out <- d1
  out[is.na(out)] <- d2[is.na(out)]
  out
}
clean_appointments <- function(x) {
  x$AppointmentID <- trimws(as.character(x$AppointmentID))
  x$PatientID <- trimws(as.character(x$PatientID))
  x$DoctorID <- trimws(as.character(x$DoctorID))
  x$Status <- tolower(trimws(as.character(x$Status)))
  x$AppointmentDate_clean <- parse_clinic_date(x$AppointmentDate)
  x$DateReview <- is.na(x$AppointmentDate_clean)
  x$ReferenceReview <- is.na(x$PatientID) | is.na(x$DoctorID)
  x
}
patient_result <- clean_patients(Patients)
Patients_clean <- patient_result$data
Patients_rejected <- patient_result$rejected
Appointments_clean <- clean_appointments(Appointments)
# Do not drop rows merely because a non-key attribute is missing.
stopifnot(nrow(Patients_clean) + length(unique(Patients_rejected$PatientID)) >=
        nrow(Patients_clean))
```

Expected evidence / interpretation

Provide reusable cleaning code, exception flags, before/after counts and a record-preservation explanation.

Cleaning issue	Automatic rule	Manual review needed?
Whitespace/case	trimws(), tolower()	No, if categories are known
Date in approved format	Parse known formats	No
Unrecognised date	Set NA and flag	Yes
Duplicate identifier	Flag all repeated IDs	Yes, unless exact duplicate policy is approved
Missing phone	Retain row and mark missing	Only if contact is required
Unknown foreign key	Retain in exception report	Yes

TASK 5 – Create a Data Reshaping Pipeline Using Melting and Casting (4 Marks)

Objective: Convert monthly branch summaries between wide and long formats and validate that totals are preserved.

Wide format is convenient for human-readable monthly reports with one column per month. Long format is generally easier for grouped analysis, plotting and modelling because month is stored as a

value in one column. Reshaping can duplicate information if identifiers are not unique or can lose information if aggregation is used without checking the grouping level.

Shared group work

- Create a wide table with branch identifiers and monthly measures.
- Use melt() to create long form and dcast()/acast() to create summaries.
- Compare results with base R reshape() where practical.
- Validate row counts, totals, identifiers and missing-value handling.

Student A – individual responsibility

- Implement wide-to-long melting and explain identifier versus measured variables.
- Choose long format for grouped analysis and plotting.

Student B – individual responsibility

- Implement casting and aggregation.
- Reconcile totals and explain when wide versus long is suitable.

R implementation

```
library(reshape2)
branch_wide <- data.frame(
  BranchID=c("B01","B02","B03"),
  Jan=c(120,90,70), Feb=c(130,95,80), Mar=c(125,105,85)
)
branch_long <- melt(branch_wide, id.vars="BranchID",
                    variable.name="Month", value.name="Appointments")
branch_cast <- dcast(branch_long, BranchID ~ Month,
                     value.var="Appointments", fun.aggregate=sum)
original_total <- sum(branch_wide[, c("Jan","Feb","Mar")])
long_total <- sum(branch_long$Appointments)
stopifnot(original_total == long_total)

# Department-by-month example
dept_wide <- data.frame(Department=c("General","Cardiology"),
                        Jan=c(40,20), Feb=c(45,25), Mar=c(50,28))
dept_long <- melt(dept_wide, id.vars="Department",
                  variable.name="Month", value.name="Visits")
dept_summary <- dcast(dept_long, Department ~ Month,
                      value.var="Visits", fun.aggregate=sum)
# Base R reshape alternative
base_long <- reshape(branch_wide, direction="long",
  varying=c("Jan","Feb","Mar"), v.names="Appointments",
  timevar="Month", times=c("Jan","Feb","Mar"), idvar="BranchID")
stopifnot(sum(base_long$Appointments) == original_total)
```

Expected evidence / interpretation

Include wide and long tables, melt/cast code and total-preservation checks.

Format	Structure	Best use
Wide	One row per branch; Jan/Feb/Mar columns	Monthly management report
Long	BranchID, Month, Appointments columns	Analysis, grouping, visualisation
Cast output	One row per branch with month columns	Exporting a compact summary

TASK 6 – Design a Multi-Stage Data-Frame Merging Strategy (5 Marks)

Objective: Integrate the five clinic datasets using explicit keys and appropriate join types while detecting unmatched and duplicated relationships.

An inner join retains only matched records. A left join retains every row from the left data frame and adds matching right-side values. A full join retains rows from both sides. The assignment's appointment-level master should normally start from Appointments and use left joins to preserve every appointment. LabTests and Billing are one-to-many child tables; joining both directly can create a Cartesian multiplication when an appointment has multiple tests and multiple billing lines. Aggregate child tables first when the report requires one row per appointment.

Shared group work

- Merge Appointments with Patients and Doctors using explicit key columns.
- Compare inner and left joins; use all, all.x and all.y where appropriate.
- Merge laboratory and billing child tables carefully and detect row multiplication.
- Check key uniqueness, unmatched references, key types and column-name clashes.

Student A – individual responsibility

- Lead Appointments-to-Patients and Appointments-to-Doctors joins.
- Validate that each appointment remains represented and identify unmatched patient/doctor IDs.

Student B – individual responsibility

- Lead LabTests and Billing integration and exception analysis.
- Explain why child-table joins can multiply rows and verify expected cardinality.

R implementation

```
# Key type standardisation
for (nm in c("PatientID","DoctorID","AppointmentID")) {
  Appointments_clean[[nm]] <- as.character(Appointments_clean[[nm]])
}
Patients_clean$PatientID <- as.character(Patients_clean$PatientID)
Doctors$DoctorID <- as.character(Doctors$DoctorID)
```

```
# Join alternatives
inner_patient <- merge(Appointments_clean, Patients_clean,
                       by="PatientID", all=FALSE)
left_patient <- merge(Appointments_clean, Patients_clean,
                      by="PatientID", all.x=TRUE)
full_patient <- merge(Appointments_clean, Patients_clean,
                      by="PatientID", all=TRUE)

# Appointment-to-doctor integration
master <- merge(left_patient, Doctors, by="DoctorID",
                all.x=TRUE, suffixes=c("_appointment","_doctor"))

# Unmatched references
unmatched_patients <- Appointments_clean[
  !Appointments_clean$PatientID %in% Patients_clean$PatientID, ]
unmatched_doctors <- Appointments_clean[
  !Appointments_clean$DoctorID %in% Doctors$DoctorID, ]

# Aggregate one-to-many tables before appointment-level join
lab_by_appointment <- aggregate(
  LabID ~ AppointmentID, data=LabTests, FUN=length
)
names(lab_by_appointment)[2] <- "LabTestCount"
bill_by_appointment <- aggregate(
  Amount ~ AppointmentID, data=Billing, FUN=sum
)
master <- merge(master, lab_by_appointment,
                by="AppointmentID", all.x=TRUE)
master <- merge(master, bill_by_appointment,
                by="AppointmentID", all.x=TRUE)
master$LabTestCount[is.na(master$LabTestCount)] <- 0L
master$Amount[is.na(master$Amount)] <- 0
stopifnot(nrow(master) == nrow(Appointments_clean))
stopifnot(!anyDuplicated(master$AppointmentID))
```

Expected evidence / interpretation

Provide staged merge code, alternative join counts, unmatched-key reports and a verified master dataset.

Join	R merge setting	Record retention
Inner	all = FALSE	Only matched key values
Left	all.x = TRUE	All rows from x plus matches from y
Right	all.y = TRUE	All rows from y plus matches from x
Full	all = TRUE	All keys from both frames

TASK 7 – Build an Automated File Reading and Writing Pipeline (5 Marks)

Objective: Import source files with controlled types, validate them and export versioned outputs without accidental overwriting.

CSV is portable and easy to inspect, but it does not reliably preserve all R-specific classes such as factors and Date metadata. RDS preserves a single R object and its classes more faithfully, but is less suitable for exchange with non-R tools. A robust pipeline may export CSV for interoperability and RDS for exact R-session reuse.

Shared group work

- Demonstrate read.csv(), read.table(), read.delim() and readLines().
- Use header, sep, na.strings, colClasses and stringsAsFactors deliberately.
- Export cleaned, reshaped and master data using write.csv()/write.table() and saveRDS().
- Read outputs back and compare dimensions, types and values; use timestamped names.

Student A – individual responsibility

- Implement source import and schema/type checks.
- Demonstrate missing-file, missing-column and wrong-type handling.

Student B – individual responsibility

- Implement export, timestamped versioning and read-back checks.
- Explain CSV versus RDS trade-offs.

R implementation

```
input_dir <- file.path(project_dir, "input")
output_dir <- file.path(project_dir, "output")
dir.create(input_dir, showWarnings=FALSE, recursive=TRUE)
dir.create(output_dir, showWarnings=FALSE, recursive=TRUE)

# Example import with explicit classes
read_source <- function(path, required, classes=NULL) {
 if (!file.exists(path)) stop("Input file not found: ", path)
 x <- read.csv(path, na.strings=c("", "NA", "NULL"),
               colClasses=classes, stringsAsFactors=FALSE,
               check.names=FALSE)
 missing <- setdiff(required, names(x))
 if (length(missing)) stop("Missing required columns: ",
                           paste(missing, collapse=", "))
 x
}
# Example: IDs imported as character
# ap <- read_source("input/appointments.csv",
#   c("AppointmentID","PatientID","DoctorID"),
#   classes=c(AppointmentID="character", PatientID="character",
```

```
#           DoctorID="character"))

# Other delimited import examples
# tab_data <- read.table("input/source.txt", header=TRUE, sep="|",
#                        na.strings=c("", "NA"), stringsAsFactors=FALSE)
# delim_data <- read.delim("input/source.tsv", header=TRUE,
#                          colClasses=c(PatientID="character"))
# raw_lines <- readLines("input/source.csv", n=5)

safe_export <- function(x, stem, directory=output_dir) {
  stamp <- format(Sys.time(), "%Y%m%d_%H%M%S")
  path <- file.path(directory, paste0(stem, "_", stamp, ".csv"))
  if (file.exists(path)) stop("Refusing to overwrite: ", path)
  write.csv(x, path, row.names=FALSE, na="")
  path
}
master_csv <- safe_export(master, "clinic_master")
saveRDS(master, file.path(output_dir, "clinic_master.rds"))
check <- read.csv(master_csv, stringsAsFactors=FALSE)
check_rds <- readRDS(file.path(output_dir, "clinic_master.rds"))
stopifnot(identical(dim(check), dim(master)))
stopifnot(identical(check_rds, master))
```

Expected evidence / interpretation

Include import/export code, type checks, versioned file naming and read-back validation.

Format	Advantages	Limitations / use
CSV	Portable, human-readable, supported widely	May change classes; use explicit import rules
Delimited TXT/TSV	Supports custom separators	Separator and quoting must be specified
RDS	Preserves R object structure and classes	Primarily for R-to-R exchange

TASK 8 – Develop a Reusable R Data-Management Function Library (4 Marks)

Objective: Create reusable functions for validation, cleaning, reshaping, merging and file handling with documented return values and errors.

Functions make the pipeline maintainable because a correction to a rule can be made once and reused. A good function checks its input, avoids hidden global dependencies and returns a predictable object. stop() is appropriate for invalid input; warning() is useful when processing can continue but the user should inspect a condition.

Shared group work

- Build at least four functions and specify their inputs, defaults and outputs.
- Use stop(), warning() and stopifnot() for invalid inputs and questionable conditions.

- At least one function returns a complex list with cleaned data, rejected records and counts.
- Test valid, boundary and invalid inputs and reuse functions on another branch/reporting period.

Student A – individual responsibility

- Lead validation and cleaning functions and test independently.
- Document default arguments, return objects and invalid-input behaviour.

Student B – individual responsibility

- Lead reshaping, merging and file functions.
- Demonstrate reuse with a different branch or period.

R implementation

```
validate_frame <- function(x, required, key=NULL) {
  if (!is.data.frame(x)) stop("Input must be a data frame")
  missing <- setdiff(required, names(x))
  if (length(missing)) stop("Missing columns: ", paste(missing, collapse=", "))
  if (!is.null(key) && anyDuplicated(x[[key]]))
    warning("Duplicate values found in key: ", key)
  invisible(TRUE)
}

clean_patient_frame <- function(x, key="PatientID") {
  validate_frame(x, c(key, "PatientName"))
  x[[key]] <- trimws(as.character(x[[key]]))
  dup <- duplicated(x[[key]]) | duplicated(x[[key]], fromLast=TRUE)
  rejected <- x[dup, , drop=FALSE]
  cleaned <- x[!duplicated(x[[key]]), , drop=FALSE]
  list(data=cleaned, rejected=rejected,
       counts=data.frame(stage=c("input","clean","flagged"),
                         records=c(nrow(x),nrow(cleaned),nrow(rejected))))
}

reshape_branch <- function(x, id="BranchID") {
  if (!requireNamespace("reshape2", quietly=TRUE))
    stop("Install reshape2 before reshaping")
  if (!id %in% names(x)) stop("Identifier column is missing")
  reshape2::melt(x, id.vars=id,
                 variable.name="Month", value.name="Appointments")
}

merge_appointments <- function(ap, pat, doc) {
  validate_frame(ap, c("AppointmentID","PatientID","DoctorID"), "AppointmentID")
  validate_frame(pat, c("PatientID","PatientName"), "PatientID")
  validate_frame(doc, c("DoctorID","DoctorName"), "DoctorID")
  x <- merge(ap, pat, by="PatientID", all.x=TRUE)
```

```
  merge(x, doc, by="DoctorID", all.x=TRUE)
}

export_versioned <- function(x, stem, directory="output") {
  if (!dir.exists(directory)) dir.create(directory, recursive=TRUE)
  path <- file.path(directory, paste0(stem, "_",
      format(Sys.time(), "%Y%m%d_%H%M%S"), ".csv"))
  if (file.exists(path)) stop("File already exists")
  write.csv(x, path, row.names=FALSE)
  invisible(path)
}

# Function tests
stopifnot(isTRUE(validate_frame(Patients_clean,
                          c("PatientID","PatientName"))))
test_clean <- clean_patient_frame(Patients)
stopifnot(is.list(test_clean), is.data.frame(test_clean$data),
          is.data.frame(test_clean$rejected))
test_long <- reshape_branch(branch_wide)
stopifnot(nrow(test_long) == nrow(branch_wide) * 3)
# Invalid-input test should raise an error:
# validate_frame(123, c("PatientID"))
```

Expected evidence / interpretation

Include function library, argument/return reference and tests.

TASK 9 – Integrate, Test and Refine the Complete Data-Management Pipeline (4 Marks)

Objective: Connect import, validation, cleaning, reshaping, merging, reporting and export into one reproducible workflow.

Recommended pipeline order: import raw files → validate schema and types → standardise → clean and flag → reshape summaries → merge master entities → calculate reports → export and read back. Do not merge unvalidated identifiers because mismatched key types can produce missing matches or errors.

Shared group work

- Integrate Tasks 2–8 in the correct sequence.
- Execute the workflow from raw files to analysis-ready data frames.
- Test missing values, duplicates, invalid IDs, unmatched keys, wrong types and malformed files.
- Repeat the run with identical inputs and record failures and refinements.

Student A – individual responsibility

- Integrate import, validation and cleaning.

- Record input, corrected and rejected counts and verify consistent error handling.

Student B – individual responsibility

- Integrate reshaping, merging and export.
- Record merged/final counts and check reproducibility.

R implementation

```
run_clinic_pipeline <- function(patients, appointments, doctors,
                                labs, billing) {
 p <- clean_patient_frame(patients)
 a <- clean_appointments(appointments)
 validate_frame(doctors, c("DoctorID","DoctorName"), "DoctorID")
 validate_frame(labs, c("LabID","AppointmentID"), "LabID")
 validate_frame(billing, c("BillID","AppointmentID"), "BillID")

 x <- merge(a, p$data, by="PatientID", all.x=TRUE)
 x <- merge(x, doctors, by="DoctorID", all.x=TRUE)

 lab_count <- aggregate(LabID ~ AppointmentID, labs, length)
 names(lab_count)[2] <- "LabTestCount"
 bill_total <- aggregate(Amount ~ AppointmentID, billing, sum)
 x <- merge(x, lab_count, by="AppointmentID", all.x=TRUE)
 x <- merge(x, bill_total, by="AppointmentID", all.x=TRUE)
 x$LabTestCount[is.na(x$LabTestCount)] <- 0
 x$Amount[is.na(x$Amount)] <- 0

 list(master=x, rejected_patients=p$rejected,
     counts=list(input_appointments=nrow(appointments),
                 master_appointments=nrow(x),
                 rejected_patient_rows=nrow(p$rejected)))
}
pipeline_result <- run_clinic_pipeline(Patients, Appointments,
                                       Doctors, LabTests, Billing)
stopifnot(nrow(pipeline_result$master) == nrow(Appointments))
# Reproducibility check for deterministic transformations
pipeline_result_2 <- run_clinic_pipeline(Patients, Appointments,
                                         Doctors, LabTests, Billing)
stopifnot(identical(pipeline_result$master,
                    pipeline_result_2$master))
```

Expected evidence / interpretation

Include a pipeline execution log, test matrix, failure/refinement record and final implementation.

Test case	Input condition	Expected behaviour
T01 Valid input	All required columns and unique keys	Pipeline completes
T02 Missing optional value	Phone is NA	Patient retained; missing field recorded

T03 Duplicate key	Repeated PatientID	Duplicate flagged and retained in rejection set
T04 Unmatched patient	Appointment references P999	Appointment retained; patient fields NA
T05 Unmatched doctor	Appointment references D99	Appointment retained; doctor fields NA
T06 Wrong type	Fee imported as character	Type check fails or explicit conversion is validated
T07 Missing file	Input path absent	stop() with clear file error
T08 Malformed schema	Required ID column absent	stop() before merge
T09 Repeat run	Same data and seed	Same output values and dimensions

TASK 10 – Create an Evidence-Based Efficiency and Maintainability Evaluation (4 Marks)

Objective: Measure the pipeline at 5,000 and 500,000 appointment rows and use evidence to refine performance.

Performance results depend on CPU, memory, R version, disk speed and other running processes. Use system.time() repeatedly and compare medians. A practical refinement is to aggregate child tables before joining, avoid repeated full-data-frame copies and import only required columns. For very large data, consider data.table or a database after establishing the base-R correctness baseline.

Shared group work

- Generate synthetic datasets at both required scales with realistic patient, doctor and branch keys.
- Benchmark at least three operations: merge, reshape/cast and file read/write.
- Record elapsed time, object size, row counts and correctness.
- Create a comparison table and graph, identify bottlenecks and apply at least one refinement.

Student A – individual responsibility

- Generate scale datasets and collect baseline measurements under comparable conditions.
- Record input size, elapsed time and correctness.

Student B – individual responsibility

- Analyse bottlenecks and implement one refinement.
- Compare before/after performance and state limitations.

R implementation

```
make_scale_data <- function(n) {
 set.seed(2026)
 data.frame(
  AppointmentID=sprintf("A%07d", seq_len(n)),
  PatientID=sprintf("P%06d", sample(seq_len(max(1000, n/2)), n, TRUE)),
  DoctorID=sprintf("D%03d", sample(1:100, n, TRUE)),
  BranchID=sprintf("B%03d", sample(1:100, n, TRUE)),
  Month=sample(month.abb, n, TRUE),
  Fee=sample(seq(300, 1000, 50), n, TRUE),
```

```
  stringsAsFactors=FALSE
 )
}
benchmark_one <- function(n) {
 x <- make_scale_data(n)
 doctors <- data.frame(DoctorID=sprintf("D%03d",1:100),
                       DoctorName=paste("Doctor",1:100))
 merge_time <- system.time(
  merged <- merge(x, doctors, by="DoctorID", all.x=TRUE)
 )[["elapsed"]]
 reshape_time <- system.time({
  summary_wide <- aggregate(Fee ~ BranchID + Month, x, sum)
  summary_long <- summary_wide
 })[["elapsed"]]
 file <- tempfile(fileext=".csv")
 write_time <- system.time(write.csv(x, file, row.names=FALSE))[["elapsed"]]
 read_time <- system.time(read.csv(file, stringsAsFactors=FALSE))[["elapsed"]]
 result <- data.frame(
  rows=n, merge_seconds=merge_time,
  summary_seconds=reshape_time, write_seconds=write_time,
  read_seconds=read_time, object_MB=as.numeric(object.size(x))/1024^2,
  merged_rows=nrow(merged), read_rows=nrow(read.csv(file))
 )
 unlink(file)
 result
}
benchmark_results <- rbind(benchmark_one(5000),
                           benchmark_one(500000))
print(benchmark_results)
write.csv(benchmark_results, "benchmark_results_actual.csv",
          row.names=FALSE)
# Plot actual measured results
barplot(benchmark_results$merge_seconds,
        names.arg=benchmark_results$rows,
        xlab="Appointment records", ylab="Merge time (seconds)",
        main="Measured merge time by input size")
```

Expected evidence / interpretation

Run the benchmark script on the submission computer and paste actual outputs into the evidence table. The graphic below visualises only the 100-fold difference in requested input volume, not measured runtime.

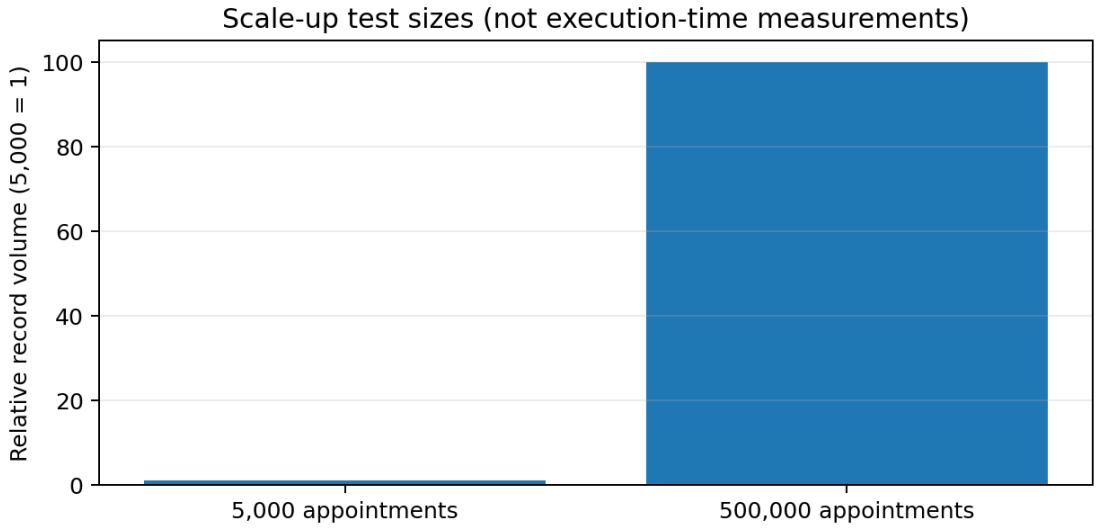


Figure 2. The required test sizes differ by a factor of 100. This is a scale illustration only. Replace it with a plot generated from benchmark_results after executing the R code.

Dataset size	Merge time (s)	Reshape/summary time (s)	Write/read time (s)	Object size (MB)	Correctness
5,000	Record from R run	Record from R run	Record from R run	Record from R run	Rows/keys verified
500,000	Record from R run	Record from R run	Record from R run	Record from R run	Rows/keys verified

Do not fill this table with guessed timings. Attach the actual console output or export benchmark_results_actual.csv as evidence.

TASK 11 – Defend the Final Pipeline Under a Changed Requirement (4 Marks)

Objective: Adapt the design for daily files from 100 branches, new doctors/departments and corrections to earlier appointments.

Use an append-only raw archive so each received file remains available for audit. Maintain a current appointment table keyed by AppointmentID and an ingestion log. For corrections, update the current view only after validating the correction and retain the prior version in history. New doctors and departments should enter a staging table, pass validation, then be added to reference tables. Unknown references should be quarantined or retained with a review flag rather than silently removed.

Shared group work

- Retain stable entity keys and add effective dates or version fields where needed.
- Handle new reference records without dropping valid appointments.
- Implement a multi-day ingestion process and an explicit correction/upsert policy.
- Test retained, updated, unmatched and newly introduced records and state remaining trade-offs.

Student A – individual responsibility

- Analyse schema evolution, keys, validation and retention.

- Explain how new doctors/departments are added without overwriting historical meaning.

Student B – individual responsibility

- Implement multi-day ingestion and correction handling.
- Test new, changed and unmatched records and compare with the original pipeline.

R implementation

```
# Simple multi-day ingestion with correction history
ingest_daily <- function(existing, incoming, received_on=Sys.Date()) {
 required <- c("AppointmentID","PatientID","DoctorID","AppointmentDate","Fee")
 validate_frame(incoming, required, "AppointmentID")
 incoming$AppointmentID <- as.character(incoming$AppointmentID)
 incoming$ReceivedOn <- as.Date(received_on)
 incoming$RecordVersion <- 1L

 if (is.null(existing) || nrow(existing) == 0) {
  incoming$RecordVersion <- 1L
  return(list(current=incoming, history=incoming))
 }

 old <- existing
 incoming$RecordVersion <- ave(
  incoming$AppointmentID, incoming$AppointmentID,
  FUN=function(z) length(z)
 )
 # Retain old rows in history; incoming row becomes current for matching IDs.
 history <- rbind(old, incoming)
 current <- old[!old$AppointmentID %in% incoming$AppointmentID, , drop=FALSE]
 current <- rbind(current, incoming)
 list(current=current, history=history)
}
# Demonstration
day1 <- Appointments_clean
day2 <- Appointments_clean[1:2, ]
day2$Fee <- day2$Fee + 50  # simulated correction
day2$AppointmentDate <- as.character(day2$AppointmentDate_clean)
daily_result <- ingest_daily(day1, day2, as.Date("2026-01-10"))
stopifnot(nrow(daily_result$current) == nrow(day1))
stopifnot(nrow(daily_result$history) == nrow(day1) + nrow(day2))
```

Expected evidence / interpretation

Show revised component, test evidence, architecture impact, limitations and future improvements.

Changed requirement	Design response	Evidence/test
Daily files from 100 branches	BranchID and ReceivedOn tracked for each batch	Ingest files from multiple branch IDs
New doctors/departments	Reference-data staging and approval	Unknown ID appears in review report

Earlier appointment corrections	Version history plus current view	Old and corrected versions both retained
Pipeline must remain extensible	Reusable functions and schema checks	Add a new field and rerun tests

TASK 12 – Document, Package and Present the Final Pipeline (4 Marks)

Objective: Package the solution so a new R session can rebuild the data frames, reports and outputs and management can understand the design.

Recommended management practice: use CSV for branch-to-branch interchange and RDS for preserving validated R objects between R sessions. Keep raw files immutable, validate schemas before processing, use left joins from the appointment table, aggregate one-to-many child records before appointment-level reporting, and maintain exception/history tables for auditability.

Shared group work

- Prepare a data dictionary with columns, types, keys, sources and validation rules.
- Create one reproducible script with set.seed(), relative paths, comments and sessionInfo().
- Create a structured summary-report function with stage counts and exception counts.
- Provide evidence-based management recommendations and complete the contribution record.

Student A – individual responsibility

- Prepare the data dictionary and run instructions.
- Verify the solution from a fresh R session.

Student B – individual responsibility

- Create the summary report and management recommendation.
- Document at least two lessons learned.

R implementation

```
make_pipeline_report <- function(result, output_files=character()) {
 stopifnot(is.list(result), is.data.frame(result$master))
 data.frame(
  Metric=c("Master appointments","Rejected patient rows",
           "Appointments with missing patient match",
           "Appointments with missing doctor match"),
  Value=c(nrow(result$master),
          result$counts$rejected_patient_rows,
          sum(is.na(result$master$PatientName)),
          sum(is.na(result$master$DoctorName))),
  stringsAsFactors=FALSE
 )
}
final_report <- make_pipeline_report(pipeline_result)
```

```
print(final_report)
cat("R version:", R.version.string, "\n")
print(sessionInfo())
```

Expected evidence / interpretation

Include data dictionary, run instructions, summary object, recommendation, contribution record and reflection.

Data frame	Column	R type	Key / relationship	Validation rule
Patients	PatientID	character	Primary key	Not missing; unique
Patients	Age	integer	—	Reasonable non-negative range
Doctors	DoctorID	character	Primary key	Not missing; unique
Doctors	DepartmentID	character	Foreign key	Must exist in Departments or be flagged
Appointments	AppointmentID	character	Primary key	Not missing; unique
Appointments	PatientID	character	Foreign key	Must match Patients or be flagged
Appointments	DoctorID	character	Foreign key	Must match Doctors or be flagged
Appointments	AppointmentDate_clean	Date	—	Must parse approved date formats
LabTests	LabID	character	Primary key	Not missing; unique
LabTests	AppointmentID	character	Foreign key	Must match Appointments or be flagged
Billing	BillID	character	Primary key	Not missing; unique
Billing	AppointmentID	character	Foreign key	Must match Appointments or be flagged

Run instructions

- Install R and RStudio, then open a new R script.
- Install reshape2 once using install.packages('reshape2').
- Copy the consolidated script from the appendix into clinic_pipeline.R.
- Run the script from the first line in a fresh R session.
- Inspect the printed validation outputs and save the benchmark table from the 5,000 and 500,000 row tests.
- Keep the raw input folder unchanged; generated outputs should be written to the output folder with timestamped names.

Contribution Record

Task	Student A – Primary contribution	Student B – Primary contribution	Joint work
Task 1	_____	_____	Review and test together
Task 2	_____	_____	Review and test together
Task 3	_____	_____	Review and test together
Task 4	_____	_____	Review and test together
Task 5	_____	_____	Review and test together
Task 6	_____	_____	Review and test together
Task 7	_____	_____	Review and test together
Task 8	_____	_____	Review and test together
Task 9	_____	_____	Review and test together
Task 10	_____	_____	Review and test together
Task 11	_____	_____	Review and test together
Task 12	_____	_____	Review and test together

Individual verification / viva preparation

Possible question	Key points to explain
Why are IDs character?	Preserve leading zeros; prevent numeric operations on identifiers.
Why use a left join from Appointments?	Preserves every appointment even when a reference is unmatched.
Why aggregate lab/billing first?	Prevents accidental many-to-many row multiplication in one-row-per-appointment reports.
What does melt() do?	Converts selected measurement columns into variable/value rows.
CSV versus RDS?	CSV is portable; RDS preserves R object classes.
How are corrections handled?	Retain old version in history; validate incoming correction; refresh current view.
How do you prove reproducibility?	Same seed, same inputs, fresh session and identical outputs.

Lessons learned

- A data frame is not safe merely because it can be created; its keys, types and relationships must be validated before joining.
- Join cardinality matters. Joining two one-to-many child tables without aggregation can inflate totals.
- Cleaning should be traceable. A flagged exception is often safer than silently deleting a record.
- Import/export settings are part of data quality because automatic type conversion can change identifiers and dates.
- Performance claims must be based on measurements from the actual execution environment.

Conclusion

The proposed clinic records pipeline demonstrates an integrated approach to R data-frame management. It creates normalised synthetic datasets, validates and cleans records, reshapes monthly summaries, merges related entities, reads and writes files safely, and packages operations as reusable functions. The design preserves appointment records through left joins, isolates unmatched references for review, and aggregates child data before appointment-level reporting to avoid unintended row multiplication. It also supports a future multi-branch workflow through batch tracking and correction history. The 5,000- and 500,000-row benchmarks must be executed on the submission computer so that the final report contains genuine timing and memory evidence.

References

17. R Core Team. R: A Language and Environment for Statistical Computing. R Foundation for Statistical Computing. https://www.r-project.org/
18. R Documentation. data.frame – Data Frames. https://stat.ethz.ch/R-manual/R-devel/library/base/html/data.frame.html
19. R Documentation. merge – Merge Two Data Frames. https://stat.ethz.ch/R-manual/R-devel/library/base/html/merge.html
20. R Documentation. read.table – Data Input. https://stat.ethz.ch/R-manual/R-devel/library/utils/html/read.table.html
21. R Documentation. write.table – Data Output. https://stat.ethz.ch/R-manual/R-devel/library/utils/html/write.table.html
22. R Documentation. reshape – Reshape Grouped Data. https://stat.ethz.ch/R-manual/R-devel/library/stats/html/reshape.html
23. Wickham, H. reshape2: Flexibly Reshape Data. R package documentation. https://cran.r-project.org/package=reshape2

Appendix A – Consolidated reproducible R script

The following compact script demonstrates the central pipeline. The task sections above contain the detailed modular code and explanations. Run the code in order and adapt output paths to the local computer.

```
# ASSIGNMENT 3 – CLINIC DATA FRAME PIPELINE
set.seed(2026)
options(stringsAsFactors = FALSE)
if (!requireNamespace("reshape2", quietly=TRUE)) install.packages("reshape2")
library(reshape2)

Patients <- data.frame(
 PatientID=c("P001","P002","P003","P004","P004"),
 PatientName=c("Asha","Bala","Charan","Divya","Divya"),
 Age=c(28L,42L,35L,51L,51L),
 Phone=c("9000000001",NA,"9000000003","9000000004","9000000004")
)
Doctors <- data.frame(DoctorID=c("D01","D02","D03"),
 DoctorName=c("Dr. Rao","Dr. Meena","Dr. Kumar"),
 DepartmentID=c("DEP1","DEP2","DEP1"))
Appointments <- data.frame(
 AppointmentID=sprintf("A%03d",1:6),
 PatientID=c("P001","P002","P003","P004","P001","P999"),
 DoctorID=c("D01","D02","D01","D03","D02","D99"),
 BranchID=c("B01","B01","B02","B02","B01","B03"),
 AppointmentDate=c("2026-01-03","03/01/2026","2026-01-05",
                   "2026-01-06","2026-01-07","2026-01-08"),
 Status=c("Completed","completed","Booked","Completed",
          "Cancelled","Completed"), Fee=c(500,600,450,700,500,300)
)
LabTests <- data.frame(LabID=c("L001","L002","L003"),
 AppointmentID=c("A001","A001","A003"),
 TestName=c("CBC","Glucose","Lipid"), ResultValue=c(12.5,98,180))
Billing <- data.frame(BillID=c("BL001","BL002","BL003","BL004"),
 AppointmentID=c("A001","A002","A003","A004"),
 Amount=c(500,600,450,700), PaymentStatus=c("Paid","Pending","Paid","Paid"))

validate_frame <- function(x, required, key=NULL) {
 if (!is.data.frame(x)) stop("Expected a data frame")
 missing <- setdiff(required,names(x))
 if (length(missing)) stop(paste("Missing:",paste(missing,collapse=", ")))
 if (!is.null(key) && anyDuplicated(x[[key]]))
   warning(paste("Duplicate key:",key))
 invisible(TRUE)
}
clean_patients <- function(x) {
```

```
 validate_frame(x,c("PatientID","PatientName"),"PatientID")
 x$PatientID <- trimws(as.character(x$PatientID))
 duplicate <- duplicated(x$PatientID) |
              duplicated(x$PatientID,fromLast=TRUE)
 list(data=x[!duplicated(x$PatientID),,drop=FALSE],
      rejected=x[duplicate,,drop=FALSE],
      input=nrow(x), output=sum(!duplicated(x$PatientID)))
}
parse_date <- function(x) {
 x <- as.character(x)
 a <- as.Date(x,format="%Y-%m-%d")
 b <- as.Date(x,format="%d/%m/%Y")
 a[is.na(a)] <- b[is.na(a)]
 a
}
clean_appointments <- function(x) {
 x$AppointmentDate <- parse_date(x$AppointmentDate)
 x$Status <- tolower(trimws(x$Status))
 x$PatientID <- as.character(x$PatientID)
 x$DoctorID <- as.character(x$DoctorID)
 x
}
p <- clean_patients(Patients)
a <- clean_appointments(Appointments)
validate_frame(Doctors,c("DoctorID","DoctorName"),"DoctorID")
validate_frame(LabTests,c("LabID","AppointmentID"),"LabID")
validate_frame(Billing,c("BillID","AppointmentID"),"BillID")

master <- merge(a,p$data,by="PatientID",all.x=TRUE)
master <- merge(master,Doctors,by="DoctorID",all.x=TRUE)
lab_count <- aggregate(LabID~AppointmentID,LabTests,length)
names(lab_count)[2] <- "LabTestCount"
bill_total <- aggregate(Amount~AppointmentID,Billing,sum)
master <- merge(master,lab_count,by="AppointmentID",all.x=TRUE)
master <- merge(master,bill_total,by="AppointmentID",all.x=TRUE)
master$LabTestCount[is.na(master$LabTestCount)] <- 0L
master$Amount[is.na(master$Amount)] <- 0
stopifnot(nrow(master)==nrow(a),!anyDuplicated(master$AppointmentID))

# Reshape and verify totals
wide <- data.frame(BranchID=c("B01","B02","B03"),
 Jan=c(120,90,70),Feb=c(130,95,80),Mar=c(125,105,85))
long <- melt(wide,id.vars="BranchID",
 variable.name="Month",value.name="Appointments")
stopifnot(sum(wide[,2:4])==sum(long$Appointments))
wide_again <- dcast(long,BranchID~Month,
```

```
 value.var="Appointments",fun.aggregate=sum)

# Safe timestamped export
out_dir <- "clinic_output"
dir.create(out_dir,showWarnings=FALSE)
out_file <- file.path(out_dir,paste0("clinic_master_",
 format(Sys.time(),"%Y%m%d_%H%M%S"),".csv"))
if (file.exists(out_file)) stop("Refusing to overwrite output")
write.csv(master,out_file,row.names=FALSE)
saveRDS(master,file.path(out_dir,"clinic_master.rds"))
read_back <- readRDS(file.path(out_dir,"clinic_master.rds"))
stopifnot(identical(master,read_back))

# Scale benchmark – run on the actual submission computer
make_scale_data <- function(n) {
 set.seed(2026)
 data.frame(AppointmentID=sprintf("A%07d",1:n),
  DoctorID=sprintf("D%03d",sample(1:100,n,TRUE)),
  BranchID=sprintf("B%03d",sample(1:100,n,TRUE)),
  Month=sample(month.abb,n,TRUE),Fee=sample(300:1000,n,TRUE))
}
benchmark <- function(n) {
 x <- make_scale_data(n)
 d <- data.frame(DoctorID=sprintf("D%03d",1:100),
              DoctorName=paste("Doctor",1:100))
 tm <- system.time(m <- merge(x,d,by="DoctorID",all.x=TRUE))[["elapsed"]]
 data.frame(rows=n,merge_seconds=tm,
           object_MB=as.numeric(object.size(x))/1024^2,
           output_rows=nrow(m))
}
benchmark_results <- rbind(benchmark(5000),benchmark(500000))
print(benchmark_results)
print(sessionInfo())
```